

Deep Reinforcement Learning

Module 1: Foundations for Reinforcement Learning

Mustafa Ozger

Aalborg University

February 24, 2026

Why this module matters (and why RL is different)

- Many engineering/science problems are **decision-making over time**.
- In RL, an **agent** learns by:
 - trying actions,
 - observing consequences,
 - improving through experience.
- Unlike typical ML, RL data is **generated by the learner's behavior**.
- Today builds the language and math behind DQN/DDQN/Actor–Critic and later MADRL.

Main message

RL is about **learning to act** to achieve long-term goals.

Learning outcomes for today

By the end of this lecture, you should be able to:

- Explain RL using the terms: **agent**, **environment**, **reward**, **policy**.
- Model a problem as an **MDP**: identify $\mathcal{S}, \mathcal{A}, P, R, \gamma$.
- Define and interpret **return** G_t and **discount factor** γ .
- Explain **value functions** V^π and Q^π and their relationship.
- Read the **Bellman equations** and explain their meaning as “consistency”.
- Describe **exploration vs exploitation** (why randomness is needed).
- Explain **on-policy vs off-policy** and **online vs offline/batch** learning at a high level.
- Understand tabular **DP, MC, TD, SARSA, Q-learning** as the foundation for deep RL.
- Know how to **evaluate** RL results correctly (seeds, train vs eval).

Agenda

- **Block A:** RL basics, Agent–Environment loop, MDPs, rewards, returns, policies
- **Block B:** Value functions, Bellman equations, backups, DP/MC/TD, SARSA/Q-learning
- **Block C:** From tabular to Deep RL + exploration + evaluation + wrap-up

How to follow

If you ever feel lost, ask: **Are we defining the problem (MDP), evaluating a policy (values), or improving behavior (control)?**

Warm-up: what does “learning” mean here?

In RL, learning means: **improving decisions from experience**.

- The agent does not get the “correct action” label.
- Instead, it gets feedback as a **reward** signal.
- The reward can be delayed (credit assignment).

Simple analogy

Learning to ride a bicycle: you do not get labeled corrections each moment, but you improve using success/failure over time.

RL in one sentence (beginner definition)

Definition (informal)

Reinforcement Learning studies **how an agent should act** in an environment to **maximize cumulative reward**.

- “Act”: choose actions at each time step.
- “Environment”: the world responds (often with uncertainty).
- “Cumulative reward”: optimize long-term outcomes, not only immediate rewards.

Key distinction

RL is not only prediction. It is **sequential decision-making under uncertainty**.

When not to use RL

RL is **not** always the right tool.

- If you already have a large labeled dataset of correct actions $\Rightarrow$ supervised learning / imitation may be easier.
- If a good physics/model-based controller exists and the model is accurate $\Rightarrow$ classical control/MPC may be better.
- RL can be data-hungry and sensitive to reward design and hyperparameters.

Rule of thumb

Use RL when **interaction is available** (simulation or safe real system) and long-term decisions matter.

A concrete motivating example: “thermostat control”

Goal: keep room temperature comfortable with minimal energy.

- State: temperature, outside temperature, time of day, occupancy (maybe).
- Action: heater power level.
- Reward: negative energy use and penalty for discomfort.
- Dynamics: temperature changes slowly and depends on disturbances.

Why RL fits

The best action depends on **future outcomes** (pre-heating, avoiding oscillations), not only current temperature.

Mini probability/expectation primer (only what we need)

We will use **expectations** because environments and policies can be random.

- A random variable X has possible outcomes with probabilities.
- The expectation is the average value under those probabilities:

$$\mathbb{E}[X] = \sum_x x P(X = x) \quad (\text{discrete case}).$$

Tiny example

If reward R is $+1$ with prob. 0.7 and -1 with prob. 0.3:

$$\mathbb{E}[R] = 1 \cdot 0.7 + (-1) \cdot 0.3 = 0.4.$$

Block A: RL basics + MDPs + returns/policies

- What problems fit Reinforcement Learning?
- RL vs supervised learning
- Agent–Environment loop and trajectory data
- Markov Decision Process (MDP): definition and assumptions
- Policies, rewards, returns and discounting

What problems fit RL? (recognizing the pattern)

RL is a good fit when you have:

- **Sequential decisions:** actions affect future states/options
- **Delayed effects:** rewards appear later (credit assignment)
- **Uncertainty:** stochasticity, disturbances, unknown dynamics
- **Interactive data:** the agent must try actions to learn
- **Measurable objective:** can be expressed via reward

Examples

Robotics, networks, energy systems, traffic control, operations/scheduling.

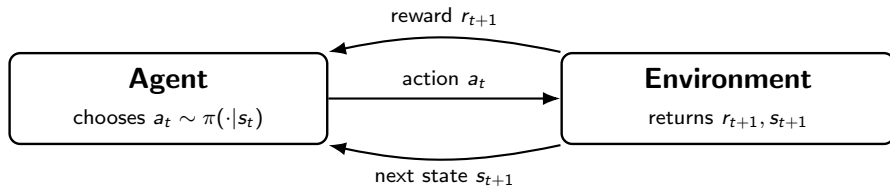
RL vs supervised vs unsupervised (what is fundamentally different?)

Supervised	Unsupervised	Reinforcement Learning
• Data: (x, y) labels • Predict y • Fixed dataset	• Data: x • Learn structure/representation • No control loop	• Data: (s, a, r, s') sequences • Maximize return • Data depends on policy

Plain-language summary

Supervised learning learns from **labels**. RL learns from **consequences**.

The Agent–Environment interaction loop (data generation)



Two important data objects

- **Trajectory:** $(s_0, a_0, r_1, s_1, a_1, r_2, \dots)$
- **Transition:** $(s_t, a_t, r_{t+1}, s_{t+1}, done)$

Trajectory vs transition (why RL data is different)

Trajectory (sequence)

$$\tau = (s_0, a_0, r_1, s_1, a_1, r_2, \dots, s_T)$$

- A full episode or segment of interaction.
- Used by Monte Carlo methods (need complete returns).
- Used by policy gradient methods (later).

Transition (one step)

$$(s_t, a_t, r_{t+1}, s_{t+1})$$

- A single learning “data point”.
- Used by TD methods (bootstrapping).
- Stored in replay buffers (Module 2).

Key mindset

RL learning is usually about turning **many correlated transitions** into useful value/policy updates.

Episode, horizon, and the role of done

- **Episode:** interaction from start state until termination (goal, failure, or time limit).
- **Horizon:** maximum number of steps (finite horizon) or potentially infinite (continuing tasks).
- In code, termination is often indicated by `done=True`.

Why done matters

When `done=True`, there is no future after the transition, so bootstrapped targets should not add $\gamma V(s_{t+1})$ or $\gamma \max Q(s_{t+1}, \cdot)$.

State vs observation (important for real systems)

- **State** s_t : complete information needed for optimal decisions (Markov).
- **Observation** o_t : what the agent sees (often partial/noisy).

Why you should care

If you only have observations, the problem may be a **POMDP** and you may need:

- history (stacked frames),
- memory (RNNs),
- belief/state estimation.

Notation cheat sheet (beginner-friendly)

- s_t : state, a_t : action, r_{t+1} : reward, s_{t+1} : next state
- $\pi(a|s)$: policy (how actions are chosen)
- $P(s'|s, a)$: transition dynamics (how the world changes)
- $R(s, a, s')$: reward function
- γ : discount factor (how far into the future we care)

Two key tasks

- **Policy evaluation:** “How good is my current policy?”
- **Control:** “How can I improve it?”

Markov Decision Process (MDP): definition and intuition

An MDP is:

$$(\mathcal{S}, \mathcal{A}, P, R, \gamma)$$

- $\mathcal{S}$: possible states
- $\mathcal{A}$: possible actions
- $P(s'|s, a)$: probability of next state
- $R(s, a, s')$: reward/feedback
- γ : discount factor

Markov property (plain language)

Given the current state and action, the future does not depend on the past.

Markov property: what it means (and what it does *not* mean)

- Markov does **not** mean deterministic. The next state can be random.
- Markov does **not** mean the past is useless. It means the past matters only through s_t .
- If your state is missing important information, the Markov property fails.

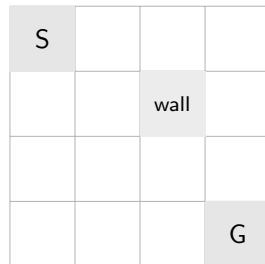
Practical rule

Design s_t (or features of o_t) so that it captures information needed for prediction and control.

Concrete MDP example: Gridworld

MDP components

- S : grid cell position (i, j)
- A : moves $\uparrow, \downarrow, \leftarrow, \rightarrow$
- R : -1 per step, $+10$ at goal
- P : deterministic move or “slip” probability
- γ : e.g. 0.95 – 0.99



Discussion prompt

What should the reward be if we want *safe* navigation (avoid obstacles) rather than only reaching the goal fast?

Rewards: what they mean (and pitfalls)

- Reward is a **score**, not a label of the correct action.
- Reward design should match the real objective.
- Pitfalls:
 - sparse reward (hard to learn),
 - loopholes (reward hacking),
 - conflicting objectives (speed vs safety vs energy).

Example

Robot: $r = +1$ for progress, -10 for collision, -0.01 per unit energy.

Return and discounting (with intuition)

Return

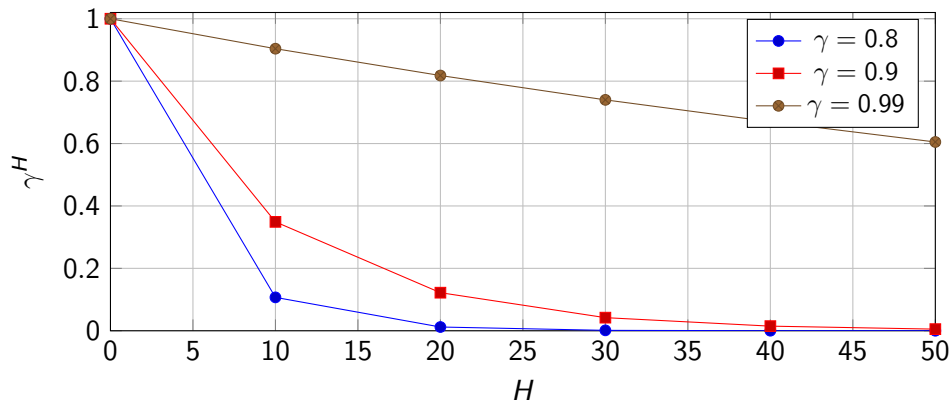
$$G_t \stackrel{\text{def}}{=} \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}.$$

- γ close to 0: focus on immediate rewards
- γ close to 1: care about long-term rewards
- Why discount?
 - mathematical stability (finite sums),
 - preference for sooner rewards,
 - continuing tasks (no terminal state).

Effective horizon: interpreting γ as “planning depth”

A practical rule of thumb: choose H so that $\gamma^H \approx 0.01$.

- After H steps, contributions $\gamma^H r$ are about 1% of immediate reward.
- Larger $\gamma \Rightarrow$ longer horizon $\Rightarrow$ more long-term planning.



Episodic vs continuing tasks

Episodic

- Terminal states exist (episodes end)
- Examples: games, navigation, many benchmarks

Continuing

- No terminal state
- Examples: temperature control, network management

Practical note

Often we turn continuing tasks into episodic ones using time limits (e.g., 1000 steps).

Policy: the agent's behavior rule

- A **policy** tells the agent what action to take in each state.
- Stochastic: $\pi(a|s)$; deterministic: $a = \mu(s)$.
- A policy can be:
 - hand-designed (baseline),
 - learned (RL),
 - learned from demonstrations (imitation learning).

Beginner question

If you had a perfect simulator, would you prefer a learned policy or a model-based planner? Why?

Policy types: deterministic vs stochastic (intuition)

Deterministic

$$a = \mu(s)$$

- One action per state.
- Exploration added externally (noise or ϵ -greedy).
- Common in some continuous-control methods (later).

Stochastic

$$a \sim \pi(\cdot|s)$$

- Policy outputs a distribution over actions.
- Exploration is built in by sampling.
- Often more robust under uncertainty.

Why this matters

Stochastic policies help exploration naturally; deterministic policies often need carefully designed exploration.

On-policy vs off-policy (high-level idea)

On-policy

- Learn the value/policy of the **same** policy that generates data.
- Example: SARSA uses a_{t+1} from the behavior policy.
- Often simpler and stable, but can be less sample-efficient.

Off-policy

- Learn about a **target policy** using data from a different behavior policy.
- Example: Q-learning uses $\max_{a'} Q(s', a')$ even if you explored.
- Often more sample-efficient, but can be less stable with approximation.

Takeaway

On/off-policy is about: **Which policy generated the data?** vs **Which policy are we improving?**

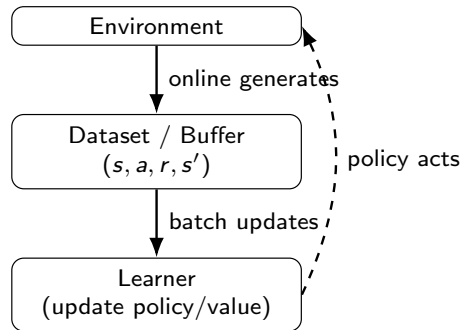
Online vs offline/batch learning in RL (conceptual)

Online RL

- The agent **interacts** with the environment while learning.
- Data is generated continuously: (s, a, r, s') .
- The policy changes over time $\Rightarrow$ data distribution changes.

Offline / batch learning

- Learn from a **fixed dataset** of transitions.
- No new environment interaction during training.
- Safer in real systems, but harder due to distribution shift (preview in Module 5).



Block B: Value functions, Bellman equations, and tabular algorithms

- Value functions: why they exist and what they mean
- Bellman equations: expectation vs optimality
- **Backups**: the basic local update used by DP/TD/Q-learning
- Model-based DP: policy evaluation, value iteration, policy iteration
- Model-free: Monte Carlo, TD learning
- Control: SARSA (on-policy), Q-learning (off-policy)

Why value functions? (intuition)

Immediate reward can be misleading.

- In many tasks, you accept short-term costs for long-term gains.
- Value functions summarize **long-term desirability**.

Two types

- $V^\pi(s)$: “How good is state s if I follow π ?”
- $Q^\pi(s, a)$: “How good is action a in state s if I follow π afterward?”

Value functions (formal definitions)

Return

$$G_t \stackrel{\text{def}}{=} \sum_{k=0}^{\infty} \gamma^k R_{t+k+1}$$

State-value and action-value

$$V^{\pi}(s) \stackrel{\text{def}}{=} \mathbb{E}_{\pi}[G_t \mid S_t = s], \quad Q^{\pi}(s, a) \stackrel{\text{def}}{=} \mathbb{E}_{\pi}[G_t \mid S_t = s, A_t = a].$$

- Expectations are over randomness in the environment and in the policy.
- In small problems: tabular arrays; in large problems: function approximation.

Relationship between V^π and Q^π

If you know Q^π , then:

$$V^\pi(s) = \sum_a \pi(a|s) Q^\pi(s, a).$$

- First sample an action using $\pi(a|s)$, then look up how good it is.
- If π is deterministic ($a = \mu(s)$), then $V^\pi(s) = Q^\pi(s, \mu(s))$.

Why this is useful

Many algorithms compute one of V or Q and derive the other when needed.

Why do we sometimes prefer Q^π over V^π ?

- Control requires choosing actions.
- If you only have $V^\pi(s)$, you still need a model to decide which action is best.
- If you have $Q^\pi(s, a)$, you can choose actions directly:

$$a = \arg \max_a Q^\pi(s, a).$$

Rule of thumb

V is great for evaluating “how good a state is.” Q is great for choosing “which action to take.”

Bellman expectation equation for V^π

$$V^\pi(s) = \mathbb{E}_{a \sim \pi(\cdot|s), s' \sim P(\cdot|s,a)} \left[R(s, a, s') + \gamma V^\pi(s') \right].$$

Expanded:

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) \left(R(s, a, s') + \gamma V^\pi(s') \right).$$

Plain language

Value now = expected reward now + discounted value later.

Bellman expectation equation for Q^π

$$Q^\pi(s, a) = \mathbb{E}_{s' \sim P(\cdot | s, a)} \left[R(s, a, s') + \gamma \mathbb{E}_{a' \sim \pi(\cdot | s')} Q^\pi(s', a') \right].$$

Interpretation

Take action a now, then behave according to π in the next state.

Bellman equations are “consistency conditions”

- Bellman equations define what a correct value function must satisfy.
- They do not say how to compute it when P and R are unknown.
- RL algorithms approximate these equations from sampled experience.

Analogy

Bellman equations are like physics equations; algorithms are numerical methods to approximate the solution.

What is a Bellman *backup*? (the local update idea)

A **backup** is one local update that replaces a value estimate with “reward now + value later”.

State-value backup (evaluation)

$$V(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) (R + \gamma V(s'))$$

Action-value backup (control)

$$Q(s, a) \leftarrow \sum_{s'} P(s'|s, a) (R + \gamma \max_{a'} Q(s', a'))$$

Old estimate
 $V(s)$ or $Q(s, a)$

backup

New estimate
reward now + value later

Backups propagate information
from future to present.

Expected backup vs sample backup (DP vs TD)

Expected backup (model-based DP)

Uses *all* next states:

$$\sum_{s'} P(s'|s, a) (R + \gamma V(s')).$$

- Needs the model (P, R) .
- Deterministic and stable in small problems.
- Often requires full sweeps over $\mathcal{S}$.

Sample backup (model-free TD)

Uses *one observed* transition (s, a, r, s') :

$$V(s) \leftarrow V(s) + \alpha (r + \gamma V(s') - V(s)).$$

- No model needed.
- Noisy updates (high variance), but online.
- Scales better when model is unknown.

Key takeaway

Both DP and TD are doing Bellman backups—DP uses expectations; TD uses samples.

Bellman operator view (optional but powerful)

Define the Bellman expectation operator for a policy π :

$$(T^\pi V)(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V(s')).$$

Then the true value function is a **fixed point**:

$$V^\pi = T^\pi V^\pi.$$

Practical meaning

Repeated backups $V_{k+1} \leftarrow T^\pi V_k$ gradually push V_k toward V^π (under standard conditions).

Why repeated backups propagate values (intuition)

- Suppose only one state near the goal gets a big reward.
- A backup updates that state's value first.
- The next backup updates states one step earlier, using the updated future value.
- Repeat $\Rightarrow$ reward information moves backward through the state graph.

Key image to remember

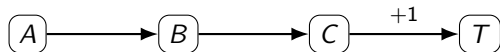
“Value at the goal leaks backward to earlier states through backups.”

Mini DP example: value propagation in a chain

Consider a chain $A \rightarrow B \rightarrow C \rightarrow T$ with reward $+1$ only at $C \rightarrow T$ and $\gamma = 0.9$.

Starting from $V_0(\cdot) = 0$:

- After 1 sweep: $V_1(C) = 1$.
- After 2 sweeps: $V_2(B) = 0.9 \cdot 1 = 0.9$.
- After 3 sweeps: $V_3(A) = 0.9^2 \cdot 1 = 0.81$.



Interpretation

Backups move reward information backward one step per sweep.

Model-based vs model-free: what information do we assume?

Model-based (DP / planning)

- We know (or learn) $P(s'|s, a)$ and $R(s, a, s')$.
- We can compute expected backups.
- Example: value iteration, policy iteration.

Model-free (learning from samples)

- We do *not* know P .
- We learn from transitions (s, a, r, s') .
- Example: MC, TD, SARSA, Q-learning.

Course structure

We learn tabular model-free methods first, then deep model-free methods (DQN/DDQN, actor-critic).

Dynamic Programming (DP): the tabular, model-known world

DP methods assume:

- small/discrete state space so we can store $V(s)$ or $Q(s, a)$ in a table,
- known model P and reward function R (or at least computable expectations).

Why DP is still important

DP gives the cleanest view of Bellman backups. Most later algorithms are noisy/approximate versions of DP.

Iterative Policy Evaluation (DP): compute V^π

Goal: evaluate a fixed policy π .

Bellman expectation backup

$$V_{k+1}(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s')).$$

- Start from an initial guess V_0 (often zeros).
- Repeatedly apply the backup until values stop changing much.

Stopping criterion

Stop when $\max_s |V_{k+1}(s) - V_k(s)| < \theta$ for small tolerance θ .

Iterative Policy Evaluation (DP): pseudocode

Algorithm (high-level)

① Initialize $V(s) \leftarrow 0$ for all s .

② Repeat:

① For each state s :

$$V_{\text{new}}(s) \leftarrow \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) (R + \gamma V(s')).$$

② Compute $\Delta \leftarrow \max_s |V_{\text{new}}(s) - V(s)|$.

③ Set $V \leftarrow V_{\text{new}}$.

③ Until $\Delta < \theta$.

What to remember

DP is “full backups” using expectations over next states.

DP convergence intuition (what does $\Delta < \theta$ mean?)

- Each sweep recomputes values using the same Bellman rule.
- If values change very little, we are close to the fixed point V^π .
- Discounting ($\gamma < 1$) helps prevent oscillations and ensures future contributions shrink.

Practical note

In code, θ is a numerical tolerance. Smaller $\theta \Rightarrow$ more iterations and slightly more accurate values.

Value Iteration (DP control): compute V^* directly

Bellman optimality backup for state values:

$$V_{k+1}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V_k(s')).$$

- This mixes evaluation and improvement: it assumes you choose the best action now.
- After convergence, recover a greedy policy:

$$\pi^*(s) \in \arg \max_a \sum_{s'} P(s'|s, a) (R + \gamma V^*(s')).$$

Value Iteration: pseudocode

Algorithm (high-level)

① Initialize $V(s) \leftarrow 0$.

② Repeat:

① For each state s :

$$V_{\text{new}}(s) \leftarrow \max_a \sum_{s'} P(s'|s, a) (R + \gamma V(s')).$$

② $\Delta \leftarrow \max_s |V_{\text{new}}(s) - V(s)|$; $V \leftarrow V_{\text{new}}$.

③ Until $\Delta < \theta$.

④ Extract greedy policy from the converged V .

Policy Iteration (DP control): evaluate $\rightarrow$ improve

Policy iteration alternates:

- 1 **Policy evaluation:** compute V^π for current π .
- 2 **Policy improvement:** make policy greedy w.r.t. the current values.

Policy improvement step

$$\pi_{\text{new}}(s) \in \arg \max_a \sum_{s'} P(s'|s, a) (R(s, a, s') + \gamma V^\pi(s')).$$

Interpretation

Policy iteration is like repeatedly asking: “How good is my policy?” then “Can I make a better choice in each state?”

Value Iteration vs Policy Iteration (how to remember)

Value Iteration

- One update mixes evaluation+improvement using max.
- Often more iterations, but each iteration is simple.
- “Always backup toward optimality.”

Policy Iteration

- Separate evaluation and improvement phases.
- Each improvement step can make big progress.
- “Evaluate then greedify.” (repeat)

Bridge to deep RL

Actor–critic methods resemble policy iteration: critic evaluates; actor improves.

Monte Carlo (MC) policy evaluation: learn from full returns

MC estimates values from complete episodes (no bootstrapping). If a state s is visited at time t , the observed return is:

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k+1}.$$

Update (incremental average form):

$$V(s) \leftarrow V(s) + \alpha(G_t - V(s)).$$

Pros/cons

- Pros: targets are real returns (no bootstrap bias).
- Cons: must wait for episode end; can have high variance.

MC update in plain language

- Run an episode, record rewards.
- For each visited state, compute the total discounted reward after that point.
- Move your estimate toward that number.

Why MC can be noisy

If episodes are long or the environment is stochastic, two runs from the same state can have very different returns.

Temporal-Difference TD(0): learn online via bootstrapping

TD(0) learns without waiting for the episode to end:

$$V(s_t) \leftarrow V(s_t) + \alpha \underbrace{(r_{t+1} + \gamma V(s_{t+1}) - V(s_t))}_{\text{TD error } \delta_t}.$$

Key idea

Replace the unknown long-term return with a 1-step estimate:

$$\text{target} = r_{t+1} + \gamma V(s_{t+1}).$$

TD error δ_t : the learning signal

$$\delta_t = r_{t+1} + \gamma V(s_{t+1}) - V(s_t).$$

- $\delta_t > 0$: outcome better than expected $\Rightarrow$ increase $V(s_t)$.
- $\delta_t < 0$: outcome worse than expected $\Rightarrow$ decrease $V(s_t)$.
- α : learning rate (how fast we update).

Why TD is so common

TD errors are the core learning signal in DQN/DDQN and actor-critic methods (later modules).

MC vs TD: bias–variance intuition

Monte Carlo

- Uses full return G_t (no bootstrap).
- Lower bias (targets are true episode outcomes).
- Higher variance (returns can be noisy).

TD(0)

- Uses bootstrap target $r + \gamma V(s')$.
- More bias (depends on current estimate).
- Often lower variance; works online.

Takeaway

MC waits and averages; TD learns immediately using a predictive “guess” of the future.

n-step returns: bridge between MC and TD

n-step target:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k+1} + \gamma^n V(s_{t+n}).$$

- $n = 1$ gives TD(0).
- Large n approaches Monte Carlo (less bootstrapping).

Why this matters

Many practical RL systems use multi-step targets to balance bias and variance.

n-step example (intuition)

Suppose $\gamma = 0.9$ and rewards are $r_{t+1} = 1$, $r_{t+2} = 1$, $r_{t+3} = 1$.

- 1-step target: $1 + 0.9V(s_{t+1})$
- 3-step target: $1 + 0.9 \cdot 1 + 0.9^2 \cdot 1 + 0.9^3 V(s_{t+3})$

Interpretation

More steps include more “real” reward information before bootstrapping.

From evaluation to control: why exploration becomes necessary

Control means: learn which actions are good.

- If you never try an action, you cannot learn its value.
- Exploration is crucial when rewards are sparse or initial estimates are wrong.
- Exploration must be balanced with exploitation (using what you know to earn reward).

Key point

Learning good control behavior requires deliberately taking some actions that are not currently believed to be optimal.

Baseline exploration: ε -greedy

- With probability ε : choose a random action (explore)
- With probability $1 - \varepsilon$: choose greedy action (exploit)

$$a_t = \arg \max_a Q(s_t, a).$$

Typical practice

Start with ε near 1.0, decay to a small value (e.g., 0.05).

Exploration schedules and GLIE (conceptual)

GLIE = “Greedy in the Limit with Infinite Exploration”.

- In theory (tabular), convergence often needs continued exploration.
- In practice, we decay exploration over time:
 - high exploration early (discover),
 - more exploitation later (refine).

Practical tip

If learning is flat, ε may be too small too early (not exploring enough).

On-policy vs off-policy revisited (connect to control)

- In control, we usually have two policies in mind:
 - **behavior policy** (how we collect data, often exploratory),
 - **target policy** (what we want to be optimal/greedy).
- On-policy: behavior = target.
- Off-policy: behavior $\neq$ target (learn optimal while exploring).

Where SARSA and Q-learning fit

SARSA: on-policy. Q-learning: off-policy.

SARSA: the idea (on-policy TD control)

SARSA learns action-values while following its current behavior policy. The name comes from the tuple:

$$(s_t, a_t, r_{t+1}, s_{t+1}, a_{t+1}).$$

Intuition

Because it uses the next action a_{t+1} that the policy actually takes (including exploration), SARSA learns the value of being slightly exploratory.

SARSA update rule

Update

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t) \right).$$

- TD target: $r_{t+1} + \gamma Q(s_{t+1}, a_{t+1})$
- TD error: $\delta = \text{target} - Q(s_t, a_t)$

What makes it on-policy

The target uses the **actual next action** drawn from the current policy.

SARSA step-by-step (how you implement it)

Given current state s :

- 1 Choose a using the current behavior policy (often ε -greedy).
- 2 Take action a , observe r and next state s' .
- 3 Choose next action a' using the same behavior policy.
- 4 Update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a)).$$

- 5 Move: $s \leftarrow s', a \leftarrow a'$.

Implementation habit

SARSA selects **two actions**: one for current step and one for next-step target.

SARSA pseudocode (high-level)

SARSA (episodic)

- ① Initialize $Q(s, a) = 0$ for all s, a .
- ② For each episode:
 - ① Initialize s ; choose a using ε -greedy from $Q(s, \cdot)$.
 - ② Repeat until terminal:
 - Take a , observe r, s' .
 - Choose a' using ε -greedy from $Q(s', \cdot)$.
 - Update $Q(s, a) \leftarrow Q(s, a) + \alpha(r + \gamma Q(s', a') - Q(s, a))$.
 - Set $s \leftarrow s', a \leftarrow a'$.

Worked SARSA update (numbers)

Assume $\gamma = 0.9$, $\alpha = 0.1$.

We observe:

$$(s, a, r, s', a') = (A, \text{Right}, 1, B, \text{Left})$$

Current values:

$$Q(A, \text{Right}) = 2.0, \quad Q(B, \text{Left}) = 3.0$$

Compute target, TD error, and update

$$\text{target} = r + \gamma Q(B, \text{Left}) = 1 + 0.9 \cdot 3 = 3.7$$

$$\delta = \text{target} - Q(A, \text{Right}) = 3.7 - 2.0 = 1.7$$

$$Q(A, \text{Right}) \leftarrow 2.0 + 0.1 \cdot 1.7 = 2.17$$

Compare to Q-learning

Q-learning would use $\max_{a'} Q(B, a')$ instead of $Q(B, \text{Left})$.

Q-learning recap: the off-policy alternative

Q-learning updates toward a greedy target:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha \left(r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t) \right).$$

- Behavior policy can be exploratory (e.g., ϵ -greedy).
- Target uses greedy next action via max.

Key distinction

SARSA uses $Q(s', a')$ (what you actually do next). Q-learning uses $\max_{a'} Q(s', a')$ (what you would do if greedy).

Worked Q-learning update (numbers)

Assume $\gamma = 0.9$, $\alpha = 0.1$ and a transition:

$$(s, a, r, s') = (A, \text{Right}, 1, B)$$

Current values:

$$Q(A, \text{Right}) = 2.0, \quad Q(B, \text{Left}) = 3.0, \quad Q(B, \text{Right}) = 1.0$$

So:

$$\max_{a'} Q(B, a') = \max(3.0, 1.0) = 3.0$$

Target and update

$$\text{target} = 1 + 0.9 \cdot 3 = 3.7, \quad Q(A, \text{Right}) \leftarrow 2.0 + 0.1(3.7 - 2.0) = 2.17$$

Notice

Same numbers as SARSA example if the next action happens to be the maximizing action.

SARSA vs Q-learning (side-by-side)

SARSA (on-policy)

$$Q \leftarrow Q + \alpha(r + \gamma Q(s', a') - Q)$$

- Learns value under behavior policy.
- Can be more conservative with risky exploration.

Q-learning (off-policy)

$$Q \leftarrow Q + \alpha(r + \gamma \max_{a'} Q(s', a') - Q)$$

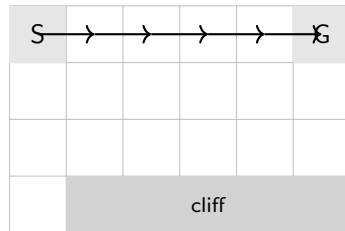
- Targets greedy behavior.
- Often learns faster in simple tabular settings.

Main takeaway

Difference is **what appears inside the target**: sampled next action vs max over next actions.

Classic example: “cliff walking” intuition (why SARSA can be safer)

- Imagine a grid where the shortest path to the goal runs next to a “cliff” (huge negative reward).
- With ϵ -greedy exploration, occasionally you take a random action.
- If you walk right next to the cliff, one random move can make you fall.



Interpretation

- SARSA learns values under exploratory behavior $\Rightarrow$ tends to avoid the cliff.
- Q-learning targets greedy policy $\Rightarrow$ may learn the shortest path close to the cliff.

Q-table: what it is and how to read it

In tabular methods, we store a number for every state-action pair:

$$Q(s, a) \quad \text{for each } s \in \mathcal{S}, a \in \mathcal{A}.$$

How to interpret one row

Fix a state s . The row $(Q(s, a_1), \dots, Q(s, a_{|\mathcal{A}|}))$ tells you which actions look promising.

$$\text{Greedy action} = \arg \max_a Q(s, a).$$

Common student confusion

$Q(s, a)$ is **not** the immediate reward. It is the **expected long-term return** starting from (s, a) .

Q-table example (before and after one update)

Assume two states A, B and two actions Left, Right. We update $Q(A, \text{Right})$.

Before

State	$Q(\cdot, \text{Left})$	$Q(\cdot, \text{Right})$
A	1.5	2.0
B	3.0	1.0

Update (Q-learning)

Given $(s, a, r, s') = (A, \text{Right}, 1, B)$, $\gamma = 0.9$, $\alpha = 0.1$:

$$\max_{a'} Q(B, a') = 3.0$$

$$\text{target} = 1 + 0.9 \cdot 3.0 = 3.7$$

$$Q(A, \text{Right}) \leftarrow 2.0 + 0.1(3.7 - 2.0) = 2.17$$

After

State	$Q(\cdot, \text{Left})$	$Q(\cdot, \text{Right})$
A	1.5	2.17
B	3.0	1.0

Tabular control summary (what you should retain)

- **Backups** are the core update mechanism (reward now + value later).
- DP uses **expected backups** with a known model (P, R).
- MC uses **full returns** (no bootstrap).
- TD uses **sample backups** with bootstrapping.
- SARSA: on-policy control (uses a_{t+1} in target).
- Q-learning: off-policy control (uses max in target).

Bridge to deep RL

DQN keeps the Q-learning target shape and replaces the Q-table with a neural network.

Block C: Deep RL + exploration + evaluation

- Why tabular methods fail (scale/continuity)
- Function approximation: neural networks for Q , V , or π
- What can go wrong (instability sources)
- Exploration: why randomness is required
- Training loop + evaluation protocol + reporting
- Module 2 preview: DQN/DDQN

Why not store a table? (scale example)

- Suppose 10 state variables, each with 100 possible values.
- Number of states: $100^{10} = 10^{20}$ (impossible to store).
- With images, the number of possible observations is astronomical.

Conclusion

We need **function approximation** to generalize across similar states.

Function approximation (replacing tables)

- Use parameterized functions:

$$Q_{\theta}(s, a), \quad V_{\theta}(s), \quad \pi_{\theta}(a|s).$$

- Neural networks can map high-dimensional inputs to values/actions.
- Training adjusts parameters θ to reduce prediction error of targets.

Intuition

A network can learn that “states that look similar” should have similar values.

Deep value learning target (conceptual)

Given a sample (s, a, r, s') :

$$y = r + \gamma \max_{a'} Q_{\theta}(s', a') \quad (\text{bootstrapped target}).$$

Loss (regression form):

$$\mathcal{L}(\theta) = \mathbb{E}[(y - Q_{\theta}(s, a))^2].$$

Important

Targets depend on the network itself $\Rightarrow$ can be unstable without additional tricks.

The “deadly triad” (why deep RL can diverge)

A classic explanation for instability is the combination of:

- **Function approximation** (neural networks generalize),
- **Bootstrapping** (targets use value estimates),
- **Off-policy learning** (data distribution differs from target policy).

Why it matters

Deep value-based RL (like DQN) contains all three $\Rightarrow$ needs stabilizers (replay buffer, target network, DDQN).

What breaks in deep RL? (beginner version)

Three major issues:

- **Moving targets** (bootstrapping): your target changes as you learn
- **Correlated samples**: sequential data violates i.i.d. assumption
- **Overestimation bias**: max over noisy values tends to be too high

Preview: solutions in Module 2

Replay buffer, target network, and DDQN are designed to reduce these issues.

Exploration in deep RL: why it is harder than tabular

- In tabular problems, exploring all actions in all states is sometimes feasible.
- In deep RL, the state space is huge (images, sensors) $\Rightarrow$ naive random actions may not reach informative states.
- Sparse reward makes exploration much harder: you might not see reward for a long time.

Practical mindset

Exploration strategy is often as important as the algorithm itself.

Training loop anatomy (what you will implement)

- Initialize networks/parameters
- Repeat:
 - select action (with exploration)
 - step environment $\rightarrow (s, a, r, s', done)$
 - store transition (optional)
 - update parameters (periodically)
- Evaluate periodically with exploration turned off
- Log curves and save checkpoints

Training vs evaluation (common beginner mistake)

- Training performance includes exploration noise $\Rightarrow$ can look worse than the policy really is.
- Evaluation should measure the learned policy:
 - turn off exploration (greedy/deterministic),
 - run multiple episodes,
 - average.

Why it matters

Otherwise you measure “policy + randomness” rather than policy quality.

Evaluation and reporting (minimum scientific standard)

- Use multiple random seeds (RL variance is high).
- Report mean $\pm$ std or confidence intervals.
- Compare algorithms fairly:
 - same environment steps,
 - same compute/network size,
 - same evaluation protocol.

Course report checklist

Environment name/version, hyperparameters, number of seeds, total steps, evaluation frequency, plots.

Debug checklist (beginner-friendly)

If learning fails, check:

- Are rewards ever non-zero? (sparse rewards can look like no learning)
- Are episodes ending correctly? (done flags, time limits)
- Are actions valid? (bounds, discretization)
- Is exploration on? (if ϵ too small, no discovery)
- Are you plotting evaluation returns, not training returns?

Sanity tests

Random policy baseline; fixed-seed reproducibility; tiny environment where you can reason about optimal behavior.

Mini-exercise: Model your domain as an RL problem

In pairs, define:

- **State/Observation:** what information is available?
- **Action:** what decisions can be made?
- **Reward:** what should be optimized?
- **Episode/horizon:** when does it end?
- **Constraints:** safety, feasibility, costs

Share

We will revisit these later when discussing which DRL algorithm fits which problem.

Summary: what you should remember

- RL: learning by interaction to maximize long-term reward.
- MDP: formal model of sequential decision-making.
- Value functions: quantify long-term desirability.
- Bellman equations and **backups**: recursive consistency behind updates.
- Tabular methods: DP (model-based), MC/TD (model-free), SARSA/Q-learning (control).
- Online vs offline/batch learning: interaction vs fixed datasets.
- Deep RL: uses function approximation; needs stability tricks.
- Exploration + correct evaluation are essential for trustworthy results.

What's next (Module 2 preview)

- Implement **DQN** (Q-learning + neural networks) with:
 - experience replay,
 - target networks.
- Implement **DDQN**:
 - reduce overestimation bias due to max.
- Learn proper experimentation:
 - multiple seeds, ablations, train vs eval curves.